PERIYAR UNIVERSITY SALEM

VIDHYAA ARTS AND SCIENCE COLLEGE

COLLEGE CODE: PER177

LegalEase

AI-Powered Legal Document Generator

Google Gemini Powered Legal Drafting Assistant

TEAM LEADER

Kavinesh kumar P

Table of Contents

Phase 1: Brainstorming & Ideation Phase

Phase 2: Requirement Analysis Phase

Phase 3: Project Design Phase

Phase 4: Project Planning Phase

Phase 5: Project Development Phase

Phase 6: Project Testing Phase

Phase 7: Project Documentation Phase

Phase 8: Project Demonstration Phase

Technology Stack

Python · FastAPI · Uvicorn · Streamlit · Google Gemini 1.5 Pro · python-docx · FPDF · Pillow · python-dotenv

PHASE 1

1. Brainstorming & Ideation Phase

Project Overview

LegalEase is an AI-powered legal document generator that simplifies the creation of legal documents through generative AI. It is built with FastAPI for the backend and Streamlit for the frontend, and uses Google Gemini 1.5 Pro to draft customizable, editable documents such as employment contracts, lease agreements and NDAs from the user's own inputs: involved parties, effective dates and key terms.

Problem Statement

Drafting a legal document usually requires legal expertise or costly professional help. Entrepreneurs, freelancers, landlords and individuals often need a contract, NDA or lease quickly but have no legal background, and starting from a blank page is slow and error-prone. They need an easy tool that turns their details into a well-structured, professionally formatted document they can edit and use.

Proposed Idea

Build a web application where the user enters a document type, the parties, the terms and an effective date, and receives an AI-generated legal document instantly. The user can preview and edit the text, then export it as a branded .PDF, .DOCX or .TXT file with a logo, formatted headings, an automatic terms table and a footer.

Core Scenarios Identified

- **Employment contract** – a startup founder enters details about a new hire, and LegalEase generates an agreement with clauses for roles, responsibilities, compensation and confidentiality. The user customizes the wording, adds the company logo and downloads a branded PDF.
- **NDA** – a freelancer specifies the parties, scope of confidentiality and effective date, and instantly receives a structured NDA.
- **Residential lease** – a landlord inputs the property address, tenant details and lease terms, and receives a formatted lease with editable sections and a downloadable .DOCX version.

Target Users

- Entrepreneurs and startup founders.
- Freelancers and professionals who need contracts and NDAs.
- Landlords, tenants and individuals without a legal background.

Expected Benefit

A single, simple interface that removes the barrier between accessibility and legal professionalism: reliable, well-formatted legal content generated in seconds, fully editable, brandable and exportable in multiple formats. Sensitive user data is handled securely.

PHASE 2

2. Requirement Analysis Phase

Functional Requirements

- Provide a web form with inputs for document type, parties involved, terms and conditions (semicolon-separated) and effective date, plus a Generate Document button.
- Generate a structured legal document with multiple sections and clauses using Gemini 1.5 Pro.
- Show the generated document in a styled HTML preview below the form.
- Allow the user to edit the generated text through an editable text area (Edit Document).
- Export the document as .TXT, .DOCX (logo, headings, terms table, footer) and .PDF (logo, header/footer on every page).
- Sanitize AI output (special characters, typographic quotes) before formatting.

Non-Functional Requirements

- Simple, clean and responsive interface built with Streamlit, with branding (logo and title).
- Modular code with separate modules for AI generation, API routes, formatting and frontend.
- Professional-grade formatting standards across all export formats.
- Gemini API key stored securely in a .env file and never shared in the code.
- Secure handling of sensitive user data.

Inputs and Outputs

Input	Description	Example
Document type	Type of legal document to generate	Freelance Work Contract
Parties involved	Names and roles of the individuals or entities	Jane Doe (Service Provider), TechNova Inc. (Client)
Terms & conditions	Specific clauses; semicolons create bullet points	Payment within 7 days of invoice; Work delivered by May 15, 2025
Effective date	Date the agreement becomes legally valid	April 15, 2025

Output: an AI-generated legal document with preview and editing, downloadable as .TXT, .DOCX or .PDF.

Pre-requisites

Requirement	Details
Python 3.10+	Installed with pip for dependency management
FastAPI & Uvicorn	Backend framework and ASGI server
Streamlit	Frontend UI framework
Google Gemini API	google-generativeai SDK and a Gemini API key
Supporting libraries	python-docx, fpdf, Pillow, requests, python-dotenv

Assumptions and Constraints

- Generation needs a valid Gemini API key and an internet connection.
- AI output may vary between runs and must be reviewed before use.
- This version runs locally; it has no user accounts or saved history.

PHASE 3

3. Project Design Phase

System Architecture

LegalEase has three main parts: the Frontend (Streamlit), the Backend (FastAPI application) and the AI layer (Google Gemini API), followed by formatting modules for DOCX, PDF and TXT. The request flow is:

```
User Interface (Streamlit)
 -> FastAPI Backend (main.py, routes.py)
 -> Document Generator (ai_core/gemini_generator.py)
 -> Gemini 1.5 Pro (Google API)
 -> Formatted Output (.txt, .docx, .pdf)
```

Model Selection

Model	Used For	Benefits
Gemini 1.5 Pro (via Google Generative AI SDK)	Legal document generation	Long context window (up to 1 million tokens), structured multi-part prompting, strong formatting and reasoning for formal documents and clauses

Module Design

Module	Responsibility
legalEaseAPI/main.py	FastAPI app, root (/) health endpoint, includes router
legalEaseAPI/routes.py	DocumentRequest Pydantic model and POST /generate endpoint
ai_core/gemini_generator.py	GeminiDocumentGenerator: builds the prompt and calls Gemini
ai_core/generator.py	Formatting helpers for document output
frontend/app.py	Streamlit interface: inputs, preview, editing, downloads
Image/	Logo.png and inverseLogo.png branding assets
config.py, .env, requirements.txt, run.sh	Configuration, API key, dependencies and startup script

API Endpoint Design

Endpoint	Purpose
GET /	Health check; confirms the service is running
POST /generate	Accepts document_type, parties, terms, dates and returns the generated document text

AI Design

- GeminiDocumentGenerator forms a structured prompt (document title, involved parties, effective date, terms and conditions) and sends it to Gemini via generate_content.

- The prompt instructs the model to ensure formal legal structure with multiple sections and legal clauses.
- The generated text is returned to FastAPI as {"document": ...}, then sanitized by the frontend before display and export.

PHASE 4

4. Project Planning Phase

Milestones

Milestone	Title	Activities
Milestone 1	Model Selection and Architecture	Select Gemini 1.5 Pro; define the architecture; set up the development environment and folder structure
Milestone 2	Core Functionalities Development	Implement document generation, output formatting, editable preview and downloads; build the FastAPI backend
Milestone 3	API Logic Integration	Create main.py; handle generation logic in routes.py with the DocumentRequest model and /generate endpoint
Milestone 4	Frontend Development	Build the Streamlit interface, HTML preview, editing and multi-format downloads
Milestone 5	Deployment	Run locally with Uvicorn and Streamlit; test all functionalities end to end

Team Details

Role	Name
Team Leader	Kavinesh kumar P

Deliverables

- Source code (FastAPI backend, Gemini generator, Streamlit frontend).
- requirements.txt, .env template and run.sh.
- Logo assets and formatting modules for DOCX, PDF and TXT.
- Screenshots of every feature in action.
- Phase-wise project report (this document).

Risks and Mitigation

Risk	Mitigation
Accuracy of AI-generated legal text	Structured prompts; editable preview so users review and correct the text before export
Special characters or typographic quotes breaking formatting	sanitize_text cleans the output before formatting
Inconsistent export formatting	Dedicated format_docx, format_pdf and format_html_preview functions; tested on all formats
Data privacy concerns	API key kept in .env; secure handling of sensitive user data
Backend not running when the UI is used	Documented start-up order: Uvicorn first, then Streamlit

PHASE 5

5. Project Development Phase

Development Flow

- The user opens the Streamlit app, enters the document type, parties, terms and effective date, and clicks Generate Document.
- The frontend sends a POST request with a JSON payload to the FastAPI /generate endpoint.
- routes.py validates the data with the DocumentRequest model and calls GeminiDocumentGenerator.generate_document(...).
- The generator sends the structured prompt to Gemini 1.5 Pro and returns the response text.
- The text is sanitized and shown as a dark-themed, scrollable HTML preview.
- The user can click Click to Edit Document to modify the text, then download it as .TXT, .DOCX or .PDF.

Core Functions

Function	Description
generate_document(...)	Builds the prompt and calls Gemini 1.5 Pro to produce the legal document
sanitize_text(text)	Removes special characters and typographic quotes for clean formatting
format_docx(text, doc_type)	python-docx: embeds the logo, adds title and paragraph formatting (Times New Roman), auto-generates a Terms table from semicolon-separated input, adds a footer
format_pdf(text, doc_type)	FPDF with custom header/footer: centered logo, bold section headings, bullet-style terms
format_html_preview(text)	Converts output to styled HTML blocks for inline display

Frontend

- Streamlit page configured with st.set_page_config (centered layout); logo centered using a three-column layout; subtle centered title "AI Legal Document Generator".
- Input fields: Document Type, Parties Involved, Terms & Conditions (semicolons for bullet points) and Effective Date.
- Dark-themed preview card, an editable text area (height 300) and three download buttons with auto-generated file names.

Technology Stack

Layer	Technology
Language / Backend	Python 3.10+, FastAPI, Uvicorn, Pydantic
Frontend	Streamlit
AI	Google Gemini 1.5 Pro (google-generativeai SDK, API key via .env)
Document export	python-docx, FPDF, Pillow

How to Run

```
pip install -r requirements.txt
# add your Gemini API key to the .env file
uvicorn legalEaseAPI.main:app --reload   # backend,  http://127.0.0.1:8000
streamlit run frontend/app.py            # frontend, http://localhost:8501
```

PHASE 6

6. Project Testing Phase

Test Approach

The backend was started with Uvicorn and the frontend with Streamlit, and every feature was tested through the browser at http://localhost:8501 using a sample Freelance Work Contract between Jane Doe (Service Provider) and TechNova Inc. (Client), effective April 15, 2025.

Functional Test Cases

No.	Test	Steps	Expected Result
1	Application loads	Open http://localhost:8501	Logo, title and the four input fields are shown
2	Generate document	Enter document type, parties, terms and date; click Generate Document	Success message and a generated legal document in the preview card
3	Edit document	Click Click to Edit Document and change the text	Editable text area opens; changes are carried into the downloads
4	Download TXT	Click Download as .TXT	Plain-text file with the document content
5	Download DOCX	Click Download as .DOCX	Word file with logo on the front page, headings, terms table and footer on the last page
6	Download PDF	Click Download as .PDF	Branded PDF with logo and footer on all pages
7	Error handling	Stop the backend or use an invalid API key	A helpful error message is shown without crashing

Manual Testing Checklist

Input and Generation

- [ ] All four input fields accept text; semicolons in terms create bullet points.
- [ ] Generate Document returns a structured document with multiple sections and clauses.

Preview, Editing and Export

- [ ] Dark-mode HTML preview is scrollable and readable.
- [ ] Edited text appears in the .TXT, .DOCX and .PDF downloads.
- [ ] Layout shows the logo and footer; bullet points and the terms table are formatted cleanly.

Errors

- [ ] A missing or invalid API key produces a handled error message.

PHASE 7

7. Project Documentation Phase

Technology Stack Summary

- Python 3.10+
- FastAPI and Uvicorn
- Streamlit
- Google Gemini 1.5 Pro API
- python-docx, FPDF, Pillow, python-dotenv

AI Integration

LegalEase uses Gemini 1.5 Pro through the Google Generative AI SDK. The GeminiDocumentGenerator class configures the API key from the environment, creates the model and sends a structured prompt built from the user's inputs. Gemini's long context window and ability to follow structured, multi-part instructions make it well suited to drafting formal documents with clear sections and legal clauses.

Security Considerations

- The Gemini API key is stored in a .env file and never shared in the code.
- Sensitive user data is handled securely.
- AI output is sanitized before it is formatted and exported.
- Pydantic validates every incoming request.

Limitations

- Generated documents may need review by a qualified legal professional before use.
- Generation needs an internet connection and a valid API key.
- No user accounts, saved history or authentication in this version.
- Currently run locally only.

Future Enhancements

- Deeper contract analysis and plain-language summaries of legal text.
- Integration with legal databases.
- Personalized recommendations for clauses and terms.
- Multilingual support to reach wider audiences.
- Cloud deployment of the backend and frontend.

Conclusion

LegalEase is a practical step towards making legal documents accessible to everyone. It combines the generative power of Gemini 1.5 Pro with a simple Streamlit interface and a modular FastAPI backend to turn a few inputs into a professional, editable legal document in seconds. Users can preview, edit and export the result as a branded .PDF, .DOCX or .TXT file. Its modular design allows easy upgrades, and the project gave valuable experience in prompt engineering, API integration, document formatting and user-centric design.

PHASE 8

8. Project Demonstration Phase

Suggested Demonstration Flow (3–5 minutes)

- **Introduction** – "LegalEase is an AI legal document generator that drafts, previews, edits and exports, powered by Gemini."
- **Start-up** – launch the FastAPI backend with Uvicorn, then the Streamlit frontend.
- **Home page** – show the logo, title and the four input fields.
- **Enter details** – Freelance Work Contract; Jane Doe (Service Provider), TechNova Inc. (Client); sample terms; April 15, 2025.
- **Generate** – click Generate Document and scroll through the preview.
- **Edit** – click Click to Edit Document and change a line of text.
- **Download** – export .TXT, .DOCX and .PDF and show the logo, footer and terms table.
- **Conclusion** – recap the flow: input -> FastAPI -> Gemini -> formatted document.

Team Summary

Team Member	Role
Kavinesh kumar P (Team Leader)	Team coordination and project lead

Closing Statement

LegalEase demonstrates an end-to-end AI-integrated web application, from simple user input, through the Gemini model, to a structured, editable and branded legal document, all delivered as a modular and documented FastAPI and Streamlit project.